How the SpongeBob Chat Project Works

A plain-language guide to the tools used in this repository and why each one is there. This is an educational, non-commercial parody project and is not affiliated with Nickelodeon.

1. The big picture

The project takes a general-purpose chat model that already understands language and teaches it to answer in the style of one character. It then wraps the result in a web page you can chat with.

```
transcripts (.txt)  ->  data_cleaner.py  ->  JSONL pairs
JSONL pairs + base model  ->  train.py / train_cpu.py  ->  LoRA adapter
base model + adapter + persona prompt  ->  app.py  ->  Streamlit chat
```

2. The base model

A large language model (LLM) is a neural network trained to predict the next piece of text (a token). Modern LLMs use the **transformer** architecture. Training on huge amounts of text gives them grammar, facts and the ability to follow instructions, but they know nothing special about SpongeBob.

This project uses open-source **instruct** models, which are already tuned to hold a conversation: **SmolLM2-360M-Instruct** on the laptop (CPU) route, and **Llama-3.2-3B-Instruct** on the Colab GPU route. The number is the parameter count: 360 million versus 3 billion. More parameters generally means better grammar, better memory of facts and better instruction following, but also more RAM and slower generation. That is why the small model sometimes repeats itself or forgets facts.

3. Fine-tuning and why LoRA

Fine-tuning continues training a model on your own examples so its default behaviour changes. Doing that to every weight needs a lot of memory, because training stores gradients and optimizer state for all parameters.

LoRA (Low-Rank Adaptation) avoids this. The original weights stay frozen. For selected weight matrices W, LoRA learns a small correction made of two thin matrices A and B, so the model uses W + B x A. The size of those thin matrices is set by the **rank r** (8 on CPU, 16 on Colab). Only these small matrices are trained, typically around 1 percent of the parameters or less.

Benefits here: it fits on a free Colab T4 or even a laptop; the saved adapter is only a few megabytes, so it is easy to download and share without copying the base model; and you can swap adapters on the same base model. **lora_alpha** scales how strongly the correction is applied, and **target_modules** lists which layers get adapters (attention projections q, k, v, o, and on Colab also the MLP layers).

4. Tools used and why

Tool	What it does	Why it is used here
PyTorch	Tensor maths and automatic gradients	The engine underneath every other library
Transformers (Hugging Face)	Loads models and tokenizers, runs generation	Standard way to load open models and apply their chat template
PEFT	Adds LoRA adapters to a model; saves and loads them	Implements LoRA so only a tiny adapter is trained
TRL (SFTTrainer)	Supervised fine-tuning loop for chat data	Handles batching and training on the Colab route

Tool	What it does	Why it is used here
Unsloth	Optimised kernels and memory use for fine-tuning	Much faster with far less GPU memory, so a 3B model fits on a free T4. Needs an NVIDIA GPU
bitsandbytes (4-bit)	Quantization: stores weights in 4 bits instead of 16 or 32	Shrinks the base model so it fits in 16 GB of GPU memory
Datasets	Reads JSONL and maps it to training text	Simple loading and preprocessing of the dialogue pairs
Accelerate	Device handling for training	Dependency of Transformers training
Streamlit	Python framework for web apps	Builds the chat page with little code; supports streaming output
data_cleaner.py	Turns Speaker: line transcripts into instruction/response pairs	The model needs prompt and reply pairs, not raw scripts
Google Colab / ngrok	Free cloud GPU; tunnel for a public URL	Offloads training from the laptop; ngrok lets the Colab-hosted app be opened in a browser

5. The persona: system prompt, chat template and few-shot examples

The **system prompt** (in `training/persona.py`) tells the model who it is and lists fixed facts, for example that SpongeBob already works at the Krusty Krab. It is used both when training and when chatting, so the model sees the same framing in both.

A **chat template** turns the list of messages (system, user, assistant) into the exact special-token format the model was built for. Using the wrong format makes a model behave badly, which is why the code always calls the tokenizer's template.

Few-shot examples are a few made-up exchanges inserted before the real conversation. Small models copy examples more reliably than they obey written rules, but they can also copy them too closely, which is why the examples avoid repeating a full introduction.

Chat memory: language models have no memory of their own. Each time you send a message, the app sends the system prompt, the examples and the last 12 messages again, and the model reads it all as new input. The history is also saved to `data/chat_history.json` so it survives a page refresh. Models can only read a limited amount of text (the context window), which is why older messages are dropped.

6. Generating replies: decoding settings

Setting	Meaning	Value in app
temperature	Higher means more random and creative; lower means safer and more repetitive	0.5 default
top_p	Only sample from the most likely tokens that add up to this probability	0.9
repetition_penalty / no_repeat_ngram_size	Discourage reusing the same words and 3-word phrases	1.25 / 3
max_new_tokens	Cap on reply length; shorter is faster	80 default
Streaming (TextIteratorStreamer)	Shows tokens as they are generated instead of waiting for the full reply	Used in the chat view

7. Why quality and speed are limited, and how to improve them

Generation speed on a CPU is low because every token requires running the whole network. Quality is limited by three things: model size, amount of training data (a few hundred lines is little; thousands is better), and data cleanliness. The most effective improvements are training on Colab with the 3B model and a few thousand real dialogue pairs, then optionally exporting a quantized version (for example for Ollama) for faster local chat.

8. Future direction

This module is intended as the dialogue engine of a larger real-time multimodal agent. A vision layer would capture the user's screen and pass a description to the model as extra context, so the character can comment on what the user is doing. The persona file and the adapter are the interface that agent would call.

9. Data and licensing note

Show transcripts and artwork are copyrighted. The repository includes none of them; you supply your own for private use, and the .gitignore keeps transcripts, model weights and chat logs out of Git. Do not publish or monetize the model or app.